

EDA e Dashboard Interativo com Streamlit

Tema: Bens declarados dos candidatos do Amazonas (Eleições 2026) **Baseado na Aula 3:** Data Storytelling e Dashboards Web Interativos

1. Contexto

Todo candidato a um cargo eletivo é obrigado, no momento do registro da candidatura, a declarar seus bens à Justiça Eleitoral. Esses dados são públicos e disponibilizados pelo TSE no Portal de Dados Abertos. Eles são um instrumento de **transparência**: permitem à sociedade acompanhar o patrimônio de quem se candidata.

Neste trabalho, você vai assumir o papel de um analista de dados encarregado de **transformar esses arquivos brutos do TSE numa história de dados navegável** — exatamente o percurso da Aula 3: da análise exploratória (entender o dado) à análise explanatória (comunicar o achado num dashboard). Você vai **cruzar** o patrimônio declarado com os atributos do candidato (partido, cargo, gênero) para tornar a análise e os filtros do dashboard muito mais ricos.

2. Fonte de dados

Conjunto **"Candidatos - 2026"** do TSE — você usará **dois** recursos e fará o **join** entre eles: <https://dadosabertos.tse.jus.br/dataset/candidatos-2026>

1. **"Bens de candidatos"** — arquivo nacional `bem_candidato_2026.zip`; use só o do Amazonas: `bem_candidato_2026_AM.csv`. Traz o **patrimônio** (um bem por linha), mas **não** traz partido, cargo nem gênero.
2. **"Candidatos"** — arquivo nacional `consulta_cand_2026.zip`; use só o do Amazonas: `consulta_cand_2026_AM.csv`. Traz os **atributos do candidato**: `SG_PARTIDO`, `DS_CARGO`, `DS_GENERO`, `DS_GRAU_INSTRUCAO`, `DS_COR_RACA`, `NM_URNA_CANDIDATO`, etc.

A chave que liga os dois arquivos é `SQ_CANDIDATO`.

- Parâmetros de leitura (atenção — padrão do TSE, **não** é o CSV "padrão"): `separador = ";"`, `encoding = "latin-1"`. No arquivo de **bens**, use também `decimal = ","`.

Dica de carga (obrigatório usar estes parâmetros):

```
bens = pd.read_csv("bem_candidato_2026_AM.csv", sep=";",  
                  encoding="latin-1", decimal=",", dtype={"SQ_CANDIDATO": str})  
cand = pd.read_csv("consulta_cand_2026_AM.csv", sep=";",  
                  encoding="latin-1", dtype={"SQ_CANDIDATO": str})
```

Sem `decimal=", "` no arquivo de bens, a coluna de valor vem como texto e todas as contas de patrimônio ficam erradas. Leia `SQ_CANDIDATO` como texto (`dtype=str`) nos dois arquivos.

Colunas relevantes:

Arquivo	Coluna	Significado
bens	<code>SQ_CANDIDATO</code>	Identificador único do candidato (chave do join)
bens	<code>DS_TIPO_BEM_CANDIDATO</code>	Descrição do tipo de bem (ex.: "Apartamento", "Veículo...")
bens	<code>VR_BEM_CANDIDATO</code>	Valor declarado do bem, em R\$
bens	<code>SG_UF</code>	Deve ser sempre <code>AM</code>
candidatos	<code>SQ_CANDIDATO</code>	Identificador único do candidato (chave do join)
candidatos	<code>DS_CARGO</code>	Cargo disputado (ex.: "GOVERNADOR", "DEPUTADO ESTADUAL")
candidatos	<code>SG_PARTIDO</code>	Sigla do partido
candidatos	<code>DS_GENERO</code>	Gênero do candidato
candidatos	<code>NM_URNA_CANDIDATO</code>	Nome na urna

Dois cuidados importantes com o dado.

1. No arquivo de **bens**, cada linha é **um bem** — um candidato aparece em várias linhas. Agregue por `SQ_CANDIDATO` **antes** do join.

2. No arquivo de **candidatos**, pode haver **SQ_CANDIDATO repetido** (mais de uma linha por candidato). **Deduplique por SQ_CANDIDATO** antes do join, ou o patrimônio será contado em duplicidade.

Desde 2022 o TSE fornece apenas uma **descrição genérica** de cada bem (por LGPD) — isso não afeta o trabalho.

3. O que você deve entregar

Dois arquivos, exatamente:

1. **analise.ipynb** — notebook Jupyter com a **Análise Exploratória (EDA)** e a **geração de um CSV limpo**.
2. **app.py** — o **dashboard Streamlit** que consome o CSV gerado.

Nada além disso é exigido. Não é necessário fazer deploy, machine learning, nem juntar com outras bases.

Parte A — Notebook de EDA (**analise.ipynb**)

O notebook deve, na ordem, com **células de texto (Markdown)** explicando cada etapa:

A1. Carga e inspeção inicial.

- Ler **os dois** CSVs com os parâmetros corretos (seção 2).
- Mostrar **shape**, **head()** e **dtypes** de cada um. Confirmar que **VR_BEM_CANDIDATO** é numérico.

A2. Limpeza, qualidade e de-duplicação.

- Verificar valores ausentes (**isna**) nas colunas relevantes.
- Confirmar que os registros são do AM (**SG_UF**).
- **Deduplicar o arquivo de candidatos por SQ_CANDIDATO** (uma linha por candidato) antes do join, justificando por quê.

A3. EDA univariada (revisão da aula anterior + storytelling desta aula), com **pelo menos 3 visualizações**:

- Distribuição do **valor dos bens (VR_BEM_CANDIDATO)** — histograma e/ou boxplot. Comente a assimetria (a distribuição é concentrada? há outliers?).
- **Tipos de bem mais comuns e/ou mais valiosos (DS_TIPO_BEM_CANDIDATO)** — gráfico de barras.

- Estatísticas descritivas (`describe()`) com um parágrafo interpretando os números (média vs. mediana, o que a diferença revela).

A4. Agregação por candidato + JOIN + geração do CSV limpo.

- Agrupar o arquivo de **bens** por `SQ_CANDIDATO` para obter, por candidato: **patrimônio total** (soma de `VR_BEM_CANDIDATO`), **quantidade de bens** e o **tipo de bem principal** (o mais frequente).
- Fazer o **join** desse agregado com os atributos do candidato (`SG_PARTIDO`, `DS_CARGO`, `DS_GENERO`, e ao menos mais um, ex.: `DS_GRAU_INSTRUCAO` ou `NM_URNA_CANDIDATO`) usando `SQ_CANDIDATO` como chave. Escolha e justifique o tipo de join (dica: `how="left"` a partir dos candidatos preserva quem declarou **0 bens**).
- Salvar o resultado em **bens_am_por_candidato.csv** (este é o CSV que o `app.py` vai usar). Ele deve conter, no mínimo: `SQ_CANDIDATO`, `NM_URNA_CANDIDATO`, `DS_CARGO`, `SG_PARTIDO`, `DS_GENERO`, `patrimonio_total`, `qtd_bens`, `tipo_principal`.
- Mostrar as primeiras linhas do CSV gerado, ordenado por patrimônio.

A5. EDA robusta habilitada pelo join (pelo menos 2 comparações entre grupos):

- Ex.: patrimônio (mediana) **por cargo**; patrimônio **por partido**; distribuição **por gênero**. Use o gráfico adequado (barras para comparar grupos, boxplot para distribuição) e comente cada achado em 1–2 frases.

A6. Narrativa (storytelling).

- Uma célula de texto final com a estrutura **contexto** → **conflito/insight** → **conclusão** (Aula 3), respondendo: *o que este conjunto de dados conta sobre o patrimônio dos candidatos do AM?* Aponte ao menos **um** achado concreto sustentado pelos seus números **que dependa do join** (ex.: diferença de patrimônio entre cargos ou partidos, concentração por grupo).

Parte B — Dashboard Streamlit (`app.py`)

O dashboard deve consumir **bens_am_por_candidato.csv** (e pode também reler o CSV bruto do AM se quiser gráficos por tipo de bem). Requisitos, todos vistos na Aula 3:

B1. Configuração e carga.

- `st.set_page_config(...)` como **primeira** chamada Streamlit (com `layout="wide"`).

- Função de carga decorada com `@st.cache_data`.
- Título e uma legenda/descrição (`st.title`, `st.caption/st.markdown`).

B2. KPIs com `st.metric`.

- Pelo menos **3 KPIs** exibidos lado a lado com `st.columns`, por exemplo: total declarado, nº de candidatos, patrimônio médio e/ou mediano.
- Pelo menos **um** KPI deve usar o parâmetro `delta` com um valor **calculado** (ex.: variação do recorte filtrado vs. a média geral). **Não** use delta "chumbado"/simulado.

B3. Filtros interativos na barra lateral (`st.sidebar`).

- Pelo menos **dois** `st.multiselect` usando as colunas do join — por exemplo **DS_CARGO** (cargo) e **SG_PARTIDO** (partido).
- Pelo menos **mais um** filtro de tipo diferente (ex.: `st.slider` de patrimônio mínimo, ou `st.radio/st.selectbox` de gênero).
- Os KPIs e gráficos devem **reagir** aos filtros (usar o DataFrame filtrado).

B4. Gráficos interativos (Plotly Express) organizados.

- Pelo menos **2 gráficos** relevantes, sendo **pelo menos um que use uma coluna do join** (ex.: patrimônio mediano por cargo ou por partido; ranking de candidatos colorido por partido; distribuição por gênero).
- Organize a tela com `st.tabs` ou `st.columns` (não empilhe tudo verticalmente sem estrutura).

B5. Exportação.

- Um `st.download_button` que permita baixar o recorte atualmente filtrado em CSV.

4. Restrições e boas práticas (valem nota — ver gabarito)

- O `app.py` roda com `streamlit run app.py` sem erros, assumindo que os CSVs estão na pasta.
- O notebook **executa de cima a baixo** sem erros (`Kernel → Restart & Run All`).
- Aplique os **princípios de design** da aula: hierarquia (KPIs no topo), clareza (sem poluição), contexto (KPIs com comparação), rótulos legíveis, uso intencional de cor.
- Comente o código minimamente e escreva as células de texto em português.

5. Entrega

Envie um `.zip` contendo `analise.ipynb`, `app.py` e o `bens_am_por_candidato.csv` gerado. Não precisa incluir os CSVs brutos do TSE. Inclua um `requirements.txt` (`streamlit`, `pandas`, `plotly`) — opcional, mas recomendado.

Peso: 10,0 pontos (distribuição detalhada no gabarito).

Anexo — Trecho para baixar e extrair os arquivos do AM (pode usar)

```
import requests, zipfile, io, pandas as pd
```

```
def baixar_am(url, sufixo="_AM.CSV"):
    r = requests.get(url, headers={"User-Agent": "Mozilla/5.0"}, timeout=180)
    z = zipfile.ZipFile(io.BytesIO(r.content))
    nome = [n for n in z.namelist() if n.upper().endswith(sufixo)][0]
    z.extract(nome, ".")
    print("Extraído:", nome)
    return nome
```

```
base = "https://cdn.tse.jus.br/estatistica/sead/odsele"
```

```
arq_bens = baixar_am(f"{base}/bem_candidato/bem_candidato_2026.zip")
```

```
arq_cand = baixar_am(f"{base}/consulta_cand/consulta_cand_2026.zip")
```

```
bens = pd.read_csv(arq_bens, sep=";", encoding="latin-1", decimal=".",
                  dtype={"SQ_CANDIDATO": str})
```

```
cand = pd.read_csv(arq_cand, sep=";", encoding="latin-1", dtype={"SQ_CANDIDATO": str})
print(bens.shape, cand.shape)
```